

Last week:

* Introduction to Isabelle's interface

• Terms / theorems

* Reminder of basic proof / reasoning styles

• FW • BW • induction • subst • cases

pen and paper ↙ Isabelle ↘

This video:

* How to define recursive functions on lists
→ theory
→ Isabelle

* Intuition^{xs}
→ a list is either Nil ([])
or Cons x xs (x :: xs)

[]
 $a :: [] \approx [a], [a, b, c] \approx a :: b :: c :: []$

→ used to represent arrays

→ used to represent a mapping
• a.k.a. association list

• E.g. [(Bob, 23), (Alice, 34), ...]

→ lists are always finite

• I.e. any list is a finite applications of Cons
 $x_1 :: x_2 :: \dots :: x_n :: []$

→ Functions on lists

• E.g. $\sum xs$, $\max xs$, $\text{map } f \ xs$

median xs
↑
Very important & crucial for functional programming
returns a list whose members are xs, with f applied to them

* Examples of recursive functions.

→ f is recursive in list, if given a list, f can be evaluated in terms of a part of the tail of that list

• I.e. f calls itself

→ A fully-defined recursive function should cover all values of the input

→ Sum :: nat list ⇒ nat
signature

Sum [] = 0

Sum (x :: xs) = x + Sum xs

Evaluation: Sum [0, 1, 3] = 0 + Sum [1, 3]

= 0 + (1 + Sum [3]) = 0 + (1 + (3 + Sum []))
= 0 + (1 + (3 + 0)) =

→ append :: 'a list ⇒ 'a list ⇒ 'a list

append [] ys = ys

append (x :: xs) ys = x :: (append xs ys)

append [1, 2] [3] = 1 :: append [2] [3]

1 :: (2 :: append [] [3]) = 1 :: (2 :: [3]) = [1, 2, 3]

* Note: 1. to guarantee termination, all recursive calls have to be done on smaller arguments

2. Only terminating function will have "simp" generated.